Kexec Handover Performance Improvements

From Stateful Serialization to a Scalable, Stateless KHO Design

Jason Miu

Google

Dec 8, 2025



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

What is Kexec Handover (KHO)

- Mechanism to persist memory and host state across “kexec” transitions.
- Introduced in Kernel v6.16.
- Key Use Cases:
  - Live Update: Updating host kernel without losing guest VM state.
  - Driver State: Persisting DMA buffers and device contexts.

The Bottleneck: Stateful KHO & Serialization

- The Current Mainline Design:
- Stateful Tracking: Subsystems register memory -> Tracked in XArrays.
  - An intermediate data structure has to be created for tracking the registered memory
- The "Finalize" Phase:
  - Explicit "Finalize" call required before kexec.
  - Serialization: Kernel linearly walks all tracking structures to serialize metadata into a linked-list
  - The linked-list is then being passed to the next kernel by FDT.
- The Problem:
  - O(N) Complexity: Serialization cost scales with memory size/fragmentation.
  - Rigid State Machine: "Finalized" state locks down the system, complicating cancellation/retry.

The Solution: Stateless KHO & Radix Tree

- Moving to a zero-copy, stateless design (patch set v3)
- Concept: The tracking structure **is** the handover structure. No conversion step.
- KHO Radix Tree:
  - Replaces internal tracking structures.
  - Handover: Just pass the tree root physical address in the FDT.
- Impact:
  - O(1) Handover: Cost is constant regardless of memory size.
  - Stateless: System is always "ready" to kexec. No explicit finalization.

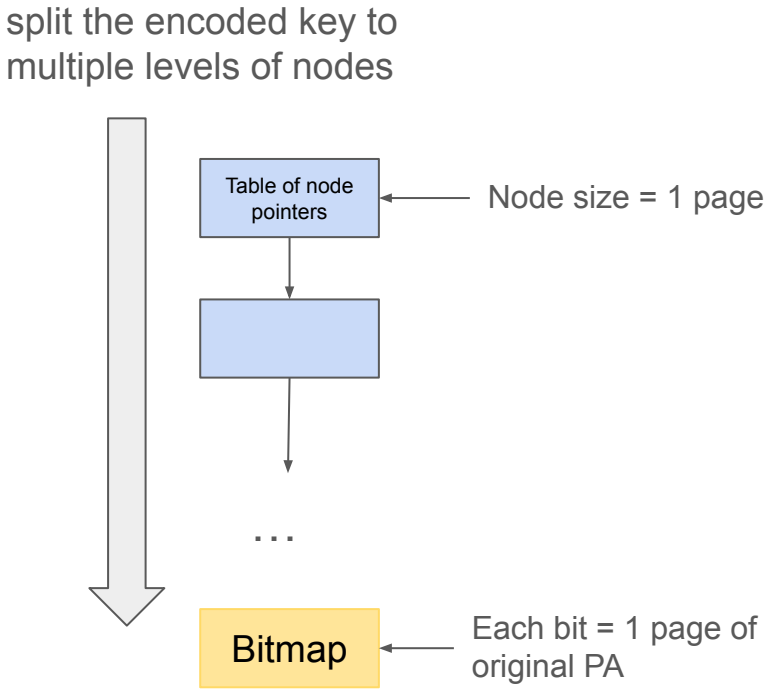
Deep Dive: Key Encoding & Tree Structure

- Handling Multi-Order Pages efficiently
- How to store different page orders in the same tree without big overhead?
- Key Encoding: Key = (Order Bit) | (Page Offset)
  - Order Bit: Encodes the size (e.g., bit 52 for Order 0).
  - Page Offset: Phy Addr >> (PAGE_SHIFT + Order)

Page order	Order Bit	Page Offset
0	...000100 ... (at bit 52)	pa >> (PAGE_SHIFT + 0)
1	...000010 ... (at bit 51)	pa >> (PAGE_SHIFT + 1)
2	...000001 ... (at bit 50)	pa >> (PAGE_SHIFT + 2)

Radix Tree Layout

- Intermediate Nodes: Tables of physical pointers to next level.
- Leaf Nodes: Bitmaps indicating preserved pages.
- Shared Paths: shares lower table levels, especially due to common zero top address bits.



The New Challenge: Boot-Time Scalability

- The "Restoration" Bottleneck.
- Current Flow (New Kernel):
  - Early Boot: KHO walks the *entire* radix tree serially.
  - For each preserved page, reserves physical memory AND initializes page metadata, e.g. page order.
- The Conflict:
  - Serial Execution: Runs on boot CPU before SMP. Slow for TBs of memory.
  - Metadata Access: Accessing `struct page` requires the `struct page` array to be initialized.
  - Deferred Init Clash: "Deferred Struct Page Initialization" leaves most page metadata uninitialized until late boot. KHO currently forces early initialization or risks invalid access.

Path Forward: Deferred Page Init & Parallel Restoration

- Parallelizing Restoration
- Decouple: Split "Physical Reservation" from "Metadata Restoration".
  - Phase 1 (Early):Reserve physical ranges in `memblock` only. No `struct page` access.
  - Phase 2 (Late/Parallel): Hook into the standard parallel init threads (`deferred_init_memmap`).
- As threads initialize memory ranges, query the KHO Tree.
- If preserved, restore metadata (page order, etc) in parallel.
- Open Discussion:
  - Tree Redesign: Current "Key" encoding optimizes storage but makes address based lookup slow (must search all orders).
  - In the leaf node bitmaps, can we mark the preserved page order instead of the preserved page?
  - Search the radix tree with all orders:
    - MAX_PAGE_ORDER * Tree depth pointer chases per look up.